ELI MKXI — Current State Snapshot (2026-07-01)

Contents

Authoritative, freshly-measured summary of the whole project as of 2026-07-01. Numbers are measured on ELI's real interpreter (`.venv/bin/python`), not the agent's system python. Deeper detail lives in the companion blueprints (`project_overview.md`, `architecture.md`, `orchestration_and_agents.md`, `grounding_and_evidence.md`, `capabilities_and_actions.md`, `security.md`, `server_frontier_roadmap.md`, `complete_findings.md`).

Scale (measured 2026-07-01)

- **143,432 LOC** across **370** `.py` files under `eli/` (plus the FastAPI web server `api/server.py`, ~3,884 lines, with an embedded dashboard PWA).
- **208** manifest capabilities (`capability_manifest.json`); **174** in the executor `SUPPORTED_ACTIONS`, plugin/route-backed remainder.
- **12 main GUI tabs** (Chat, Proactive, Images, Quick Actions, Screen, Files, Labs, Coding, Tasks, Report Builder, Eli's World, Settings — Test & Review + Orchestration are Labs sub-tabs); **14** bus agents + the CodeAgent; **4** SQLite stores (user / agent / system_index / coding_memory).
- **A second front end:** a local-first **FastAPI web server + dashboard PWA** (`api/server.py`) — chat, live telemetry, ELI's own smart-home, research corpora, the tamper-evident audit trail, and an admin console. Launchable one-click from the desktop GUI (Settings) or standalone.
- **189** test files.
- **License:** source-available — PolyForm Internal Use 1.0.0 (`LICENSE/NOTICE`); use+modify for internal/personal use, no redistribution; all commercial rights reserved. Governance: `SECURITY.md`, `CONTRIBUTING.md` (inbound grant).

Tests & eval (measured on `.venv`, 2026-07-01)

- Full run: **7,347 passed, 5 failed, 45 skipped, 2 xfailed**. The 5 reds are pre-existing and unrelated to current work — they belong to the in-progress `smart_home` plugin removal (voice SMART_HOME now uses ELI's own MQTT server) and one stale blueprint reference; they fail identically on a clean tree.
- Composition: original unit/regression/integration + **`tests/claims/`** (examines the project vs its claims: every module compiles + core imports; every manifest capability well-formed + flags match the live executor; every SUPPORTED_ACTION handled; every documented action reachable; activation phrases; blueprint refs resolve; structural/behavioural claims; **symbol inventory** = every public function/class/method is a real introspectable callable) + meta-tests for the doc pipeline, evidence planner, and test generator + generated tests.
- **Eval harness:** model-free router cases auto-run under pytest (green) + engine cases (model-backed). The engine board + full report refresh **nightly** (scheduled `eval` task), as does ELI-assisted

test generation (`testgen`).
- **2 xfail = documented, intentional:** `MOUSE_CONTROL` bare "left click" → GAZE_CLICK by design; one grounding-mode case.

GPU / inference

ELI runs on the **GPU**. `.venv/bin/python` → `llama_cpp.llama_supports_gpu_offload()` = **True** (llama-cpp-python, CUDA). The main model is loaded model-agnostically by a free-VRAM-aware adaptive loader (GPU layers → batch → ctx, reduced to fit; user-chosen ctx anchored). On the current 8 GB RTX 2060 SUPER the resident model is a large <think> MoE (`Qwen3.6-35B-A3B`, ~3B active) run partly on CPU by deliberate choice — the inference path is never hardcoded to any model name or size. Reasoning-model support (think-strip on output + stream, no-think prefill on utility calls, over-ctx head+tail truncation) is in place.

What's new since the last snapshot (2026-06-28 → 2026-07-01)

Web app / phone companion hardening

- **Stable LAN API token + Rotate button** — the token is now persisted once under `config_dir()` (`api/api_token.py`, 0600) and reused across restarts, so a paired phone is no longer stranded on a 401 every time the server restarts. A **Rotate token** control in the server panel mints a fresh token on demand (applied live to the running server) to retire paired devices deliberately.
- **Self-healing PWA** — the service worker never strands the phone on a stale shell, and the client fetch now retries a *connection-establishment* failure with backoff (the Wi-Fi power-save wake-drop), scoped so a streamed reply can never be double-delivered.
- **Frontier dashboard widgets** — live cognition DAG, autonomy goals, an overnight task scheduler, an "Ask ELI" panel, and live-vitals; richer smart-home rooms and a real app icon.

News honesty + freshness

- **Stale-cache disclosure** — when a refresh brings nothing live (offline, or the newest item is ≥2 days old) the synthesis prompt now carries a mandatory freshness disclosure; ELI states it's cached, gives the age, and is forbidden from implying a story is from "today" (it was fabricating "breaking … today" over a stale cache).
- **Interest-half recency gate** — the "articles that may interest you" surfaces (both `build_news_briefing` and `interest_news_block`) now drop any match older than a freshness window, so a two-week-old arXiv/ScienceDaily paper is never paraded as current while the top half is today's.

Learning / memory→adapter

- **Dataset builder reads the real DBs** — `USER_DB`/`AGENT_DB` now resolve from `paths.db_dir()` instead of a hardcoded `artifacts/db` path, so on any per-user or redistributed install the LoRA-consolidation dataset is built from actual data (it was silently opening empty databases and training on nothing).

What's new since the last snapshot (2026-06-19 → 2026-06-28)

A full local-first web app + dashboard (`api/server.py`)

- **Chat** with markdown + code rendering, sessions/history, stop, and regenerate; streaming via SSE on the same local GGUF and pipeline as the desktop app.
- **Live dashboard / Overview** — measured GPU/CPU/RAM vitals, model + uptime, audit integrity, device summary, recent activity, and the **monitored Internet switch**.
- **Modern responsive UI** — design tokens, light/dark themes, depth/motion, mobile; install-as-an-app (PWA manifest + service worker); runs in-process from the GUI.
- **OpenAI-compatible** `/v1/chat/completions` so standard clients connect to the local model.

ELI's own smart-home (Home Assistant fully removed)

- **ELI's own MQTT device server** (`eli/runtime/device_server.py`) replaced the Home Assistant dependency end-to-end (the HA plugin was deleted; voice SMART_HOME repoints to the native server).
- **Rooms** (group + per-room control), **scenes**, and **real automations**: trigger types time / sunrise-sunset / device-state, with **conditions** (all must hold), **multi-action**, and a **location picker** for sun triggers.
- **Network device discovery** (mDNS), **usage learning**, and ELI turning learned patterns into **suggested automations** surfaced through the proactive daemon.

Identity, roles, and the audit trail

- **RBAC** — admin / member / **viewer (read-only)** roles, authenticated identity, role-gated endpoints, and an **enterprise admin console** (audit integrity, per-user activity, the approval/risk-gate policy).
- **Tamper-evident, hash-chained audit trail** (Phase 6) — every API action recorded (who/what/outcome, metadata only); the chain is **HMAC-keyed** and live-verifiable; surfaced in the Audit tab.

Research collaboration

- Shared **corpora** with attribution, notes, and an activity feed; local ingest → FAISS → grounded, cited Q&A.

Browser voice

- **Local** whisper STT in and **local** Piper TTS out — no cloud, gated by netguard.

Security hardening (this week + 2026-06-28)

- **Secret files born locked** — new `eli/core/secure_io.py` writes the audit HMAC key, the API token store, settings, and the agent-trust registry atomically at **0600** (mkstemp → write → fsync → `os.replace`), closing the brief world-readable TOCTOU window that the old write-then-chmod left.
- **HMAC-keyed audit chain**; **fail-closed API auth** gate; **sandboxed** research ingest.
- **Monitored Internet toggle + egress oversight** — ELI stays offline-by-default and hard-gated at the socket boundary (`netguard`), but the owner can deliberately enable internet (desktop `Net` toggle or the web dashboard; web flip is **admin-only**). Both flips are audited, and — once on — `netguard` **records every allowed outbound connection** (host:port) to an in-memory live tail (`GET /v1/net/egress` + the Overview widget) and, throttled, to the tamper-evident audit ledger (`net_egress` events). The socket failsafe still governs every connection — and every connection it allows is on the record. The guard covers all four outbound paths cross-platform: `connect` / `create_connection` (raise when blocked), the non-raising `connect_ex` (returns `ENETUNREACH`), and a best-effort Windows `ProactorEventLoop` (`ConnectEx`) patch — so none silently bypasses the block or the recorder.

Pipeline / cognition fixes (2026-06-28)

- **Routing** — proof-of-reading challenges ("prove you actually read the file") route to a grounded audit by reading the *whole utterance* for the challenge frame, while the imperative "read the data in that file and prove it's right" does not; the referenced file is threaded through so the audit follows the path the user named (and a follow-up with no path uses the conversational current-file).
- **Grounded file audit** now derives its target from the referenced path (AST-grounded structural read) instead of a hardcoded GUI file.
- **SUMMARIZE_FILE never truncates** — reads the whole file and summarises via hierarchical map-reduce sized to the model's real context window (was capped at the first ~8 KB).
- **Foreground-priority broker preemption**, reasoning-model empty-turn salvage (no-think retry on both the broker and the streaming path), and **real** SELF_ANALYZE root-cause inference (tiered: grounded signatures → capped no-think LLM → per-failure description) — all verified.

Honest weaknesses (largely unchanged; from `complete_findings.md`)

- **God-files:** `executor_enhanced.py`, `engine.py` (~13–14k each) + the GUI (~11k).
- **Many swallowed `except Exception`** — failures are absorbed, not surfaced (a known observability debt; an in-tree test guards against the count growing).
- **Load-bearing `globals()`/install-time wrappers** (netguard, adaptive GGUF loader, middleware table) — architecture, not test-maskers, but a fold-in target.
- **Duplication:** two image engines; overlapping `runtime/*_response/*_surface`.
- **Repo hygiene:** some root junk / one-off scripts remain.
- **The real ceiling is the local model** — the body/memory/grounding/awareness are frontier; the *mind* is whatever local GGUF is loaded (model-agnostic → a swap).

Verdict

ELI is now a two-front local-first system: a frontier desktop cognitive assistant **and** a self-hosted web app with its own smart-home, multi-user roles, a tamper-evident audit trail, collaborative research corpora, browser voice, and a monitored path to the internet — all local, all owner-controlled. It tests itself (7,347 passing), evals itself nightly, writes its own tests, grounds its generation in real evidence, and never hardcodes its model. The open work is engineering debt (god-files, swallowed exceptions, surface duplication) and the model ceiling — both known, both phased, neither blocking.